Database Fundamentals

FOR BACKEND ENGINEERS

Master SQL, NoSQL, ACID, CAP, Normalization, Indexing, Query Optimization

Complete with Real-World Examples, Performance Tips, and Interview Answers

---

WHY DATABASE KNOWLEDGE SEPARATES JUNIOR FROM SENIOR

You can write beautiful code, but if your database queries take 10 seconds instead of 10 milliseconds, your application is unusable. Database design and optimization is the #1 performance bottleneck in backend systems.

This guide covers the fundamentals every backend engineer must master. Not theory for theory's sake — these are the concepts that come up in production incidents, code reviews, and senior-level interviews.

What You'll Learn

- SQL vs NoSQL: when to use each, and why both exist
- ACID properties: what databases guarantee, and why it matters
- CAP theorem: the fundamental trade-off in distributed systems
- Normalization: eliminating redundancy without killing performance
- Indexing: turning 10-second queries into 10-millisecond queries
- Query optimization: reading EXPLAIN plans, avoiding N+1, using JOINs correctly
- Transactions: ensuring data consistency under concurrent load
- Connection pooling: why opening a new connection per request kills your DB
- Migrations: safely evolving your schema in production without downtime

---

01 ⚖️ SQL VS NOSQL

The First Decision: Relational or Document?

🎯 The Core Question

The first architecture decision you make: SQL (relational) or NoSQL (document/key-value)? This choice affects everything: how you model data, how you query it, how it scales, and what guarantees you get.

SQL (Relational Databases)

Data stored in tables with predefined schemas. Rows and columns. Relationships via foreign keys. Query with SQL language.

```
// ── SQL: PostgreSQL Example ──────────────────────────────
// Tables defined with strict schema:

CREATE TABLE users (
  id        SERIAL PRIMARY KEY,
  email     VARCHAR(255) UNIQUE NOT NULL,
  name      VARCHAR(100) NOT NULL,
  created_at TIMESTAMP DEFAULT NOW()
);

CREATE TABLE posts (
  id        SERIAL PRIMARY KEY,
  user_id   INTEGER REFERENCES users(id),  -- Foreign key!
  title     VARCHAR(200) NOT NULL,
  content   TEXT,
  published  BOOLEAN DEFAULT false
);

// Querying with JOINs:
SELECT users.name, posts.title
FROM users
JOIN posts ON users.id = posts.user_id
WHERE users.email = 'alice@example.com';

// Every row in "posts" must have same columns
// Foreign key ENFORCED: cannot insert post with invalid user_id
```

NoSQL (Document Databases)

Data stored as JSON documents. No fixed schema. Documents in same collection can have different fields. No JOINs — embed or reference.

```
// —— NoSQL: MongoDB Example ————————————————————————
// No schema defined — just insert documents:

// User document:
{
  _id: ObjectId("abc123"),
  email: "alice@example.com",
  name: "Alice",
  posts: [  // Embedded — no JOIN needed!
    {
      title: "My First Post",
      content: "...",
      published: true
    },
    {
      title: "Second Post",
      content: "...",
      published: false
    }
  ]
}

// Query (no JOIN):
db.users.findOne({ email: "alice@example.com" });
// Returns user + all posts in ONE document

// Documents can differ:
{ _id: "def456", email: "bob@example.com", age: 30 }  // "age" field!
{ _id: "ghi789", email: "carol@example.com" }        // No "age" field!
// MongoDB doesn't care — both valid
```

Side-by-Side Comparison

Aspect	SQL (Relational)	NoSQL (Document)
Data model	Tables, rows, columns	Collections of JSON documents
Schema	Rigid — defined upfront	Flexible — documents can differ
Relationships	Foreign keys + JOINs	Embedding or referencing
Query language	SQL (structured)	MongoDB query operators, or custom
ACID	Full ACID guarantees	Eventual consistency (most), or limited ACID
Scaling	Vertical (bigger server)	Horizontal (sharding across servers)

Best for	Complex relationships, reports	Flexible schemas, rapid iteration
Examples	PostgreSQL, MySQL, SQLite	MongoDB, DynamoDB, CouchDB

When to Choose SQL

- Complex relationships: many-to-many (users ↔ roles ↔ permissions)
- Strict data integrity: banking, payments, accounting — cannot afford inconsistency
- Rich querying: aggregations, GROUP BY, complex JOINs, reporting
- Transactional guarantees: transfer money = debit A, credit B — both or neither
- Schema stability: data structure won't change often

When to Choose NoSQL

- Flexible schema: user profiles with varying custom fields
- Denormalized data: blog posts with embedded comments — read with 1 query
- Horizontal scaling: billions of documents across many servers (sharding)
- Rapid iteration: startup where data model changes weekly
- Document-like data: JSON APIs, content management, catalogs

The Hybrid Reality

Most systems use BOTH. PostgreSQL for users/payments (strict ACID), MongoDB for logs/sessions (flexible schema). Redis for caching (key-value). The database you choose is not religion — it's a tool. Pick the right tool for each job.

Interview Answer: SQL vs NoSQL

"SQL databases use tables with fixed schemas and foreign keys for relationships. You query with JOINs. They guarantee ACID transactions — essential for financial data. NoSQL databases store flexible JSON documents. No JOINs — you embed related data. They scale horizontally and allow schema evolution without migrations. I'd choose SQL when data integrity and complex queries matter, NoSQL when I need flexible schemas and horizontal scaling. In practice, many systems use both."

02 🛡 ACID PROPERTIES

What Databases Guarantee (and Why It Matters)

🎯 What is ACID?

ACID is a set of four guarantees that a database transaction provides. These guarantees prevent data corruption, ensure consistency, and allow safe concurrent access. ACID is the reason SQL databases are trusted for financial systems.

ACID stands for: Atomicity, Consistency, Isolation, Durability. Let's break down each one with real examples.

A — Atomicity: All or Nothing

A transaction is atomic if it either completes entirely or fails entirely. No partial execution. Think of it as a single unit of work that cannot be split.

```
// —— Example: Bank Transfer ————————————————————————
BEGIN TRANSACTION;

  -- Step 1: Deduct $100 from Alice
  UPDATE accounts SET balance = balance - 100 WHERE user_id = 1;

  -- Step 2: Add $100 to Bob
  UPDATE accounts SET balance = balance + 100 WHERE user_id = 2;

COMMIT;

// If Step 2 fails (server crashes, constraint violation):
// Atomicity guarantees: Step 1 is ROLLED BACK
// Result: neither account changed. Money didn't vanish.

// WITHOUT atomicity:
// Step 1 succeeds, Step 2 fails → Alice loses $100, Bob gains $0
// $100 disappeared into the void. DISASTER.
```

Why This Matters

Imagine your payment processor debits the customer but fails to create the order. Without atomicity, you charged them for nothing. Atomicity prevents this: if order creation fails, the charge is rolled back automatically.

C — Consistency: Rules Always Enforced

Consistency means the database enforces all defined rules (constraints, foreign keys, CHECK conditions). A transaction cannot leave the database in an invalid state.

```
// —— Example: Foreign Key Constraint ————————————————
CREATE TABLE orders (
  id      SERIAL PRIMARY KEY,
  user_id  INTEGER REFERENCES users(id),  -- Foreign key
  total    DECIMAL(10,2) CHECK (total > 0) -- Must be positive
);

// Try to insert invalid order:
INSERT INTO orders (user_id, total) VALUES (999, -50);
// ERROR: Foreign key violation (user 999 doesn't exist)
// ERROR: Check constraint (total cannot be negative)

// Consistency guarantee: invalid data CANNOT enter the database
// The database rejects the transaction entirely
```

I — Isolation: Transactions Don't Interfere

Isolation means concurrent transactions do not see each other's uncommitted changes. Each transaction executes as if it were the only transaction running.

```
// —— Example: Two Concurrent Transfers ————————————————
// Initial: Alice has $100

// Transaction A (started):      Transaction B (started):
BEGIN;                           BEGIN;
  SELECT balance FROM accounts    SELECT balance FROM accounts
  WHERE user_id = 1;              WHERE user_id = 1;
  -- Sees: $100                   -- Sees: $100

  UPDATE accounts                 UPDATE accounts
  SET balance = 100 - 50          SET balance = 100 - 30
  WHERE user_id = 1;              WHERE user_id = 1;
COMMIT;                          -- Transaction A committed!
                                 -- What does B see now?

// Isolation levels determine this:
// READ COMMITTED: B sees $50 (A's committed change)
// REPEATABLE READ: B still sees $100 (snapshot isolation)
// Without isolation: race condition, lost update
```

Isolation Levels

Read Uncommitted

Dirty reads allowed — see uncommitted changes. Almost never used.

Read Committed (default)

Only see committed data. Most common level.

Repeatable Read

Snapshot of data at transaction start — queries see same data throughout.

Serializable (strictest)

Transactions execute as if serial (one after another). Slowest but safest.

D — Durability: Committed Means Forever

Once a transaction is committed, it is permanent — even if the server crashes 1 second later. The data is written to disk, not just held in memory.

```
// —— Example: Order Confirmation ————————————————————
BEGIN;
  INSERT INTO orders (user_id, total) VALUES (1, 99.99);
COMMIT;  -- Returns success to user

// 1 second later: server crashes (power failure)
// Server restarts...

SELECT * FROM orders WHERE user_id = 1;
// The order IS THERE. Durability guarantees it.

// WITHOUT durability:
// User got confirmation email, but order vanished
// They were charged, but no record exists
```

ACID in Practice

PostgreSQL (Full ACID)

All four properties guaranteed

Use transactions with BEGIN/COMMIT

Isolation levels configurable

MongoDB (Limited ACID)

Single-document ACID since v1

Multi-document transactions since v4.0

Eventual consistency in replicas

Durability via write-ahead log (WAL)

Perfect for: payments, inventory, user accounts

Durability configurable (write concern)

Trade ACID for horizontal scale

Interview Answer: ACID

"ACID is four guarantees for database transactions. Atomicity: all or nothing — if transfer fails, both debit and credit roll back. Consistency: rules enforced — foreign keys, constraints always valid. Isolation: concurrent transactions don't see each other's uncommitted changes — prevents race conditions. Durability: committed data survives crashes — written to disk, not just memory. SQL databases like PostgreSQL guarantee all four. NoSQL databases trade some ACID for horizontal scale."

---

03 📐 CAP THEOREM

The Fundamental Trade-off in Distributed Systems

🎯 The Iron Law of Distributed Databases

CAP Theorem states: in a distributed database, you can only have TWO of these three guarantees at the same time. You MUST sacrifice one.

CAP stands for: Consistency, Availability, Partition Tolerance.

C — Consistency

Every read receives the most recent write. All nodes see the same data at the same time. If you write X to node A, reading from node B immediately returns X.

A — Availability

Every request (read or write) receives a response — success or failure. The system never refuses a request, even if some nodes are down.

P — Partition Tolerance

The system continues to operate even if network messages are lost or delayed between nodes. Network failures do not bring down the system.

Why You Can't Have All Three

Imagine two database nodes: Node A and Node B. They replicate data between each other.

Scenario: Network fails. Nodes A and B cannot communicate (network partition).

User writes X to Node A. User reads from Node B.

Choice 1: Return stale data from Node B (lose Consistency, keep Availability)
Choice 2: Refuse the read until A and B reconnect (lose Availability, keep Consistency)
Choice 3: Don't build a distributed system (lose Partition Tolerance — single node)

You MUST pick two. CAP theorem proves you cannot have all three.

The Three Choices

Choice	Guarantees	Sacrifice	Example Systems
CP	Consistency + Partition Tolerance	Availability	MongoDB, HBase, Redis Cluster
AP	Availability + Partition Tolerance	Consistency	Cassandra, DynamoDB, CouchDB
CA	Consistency + Availability	Partition Tolerance	PostgreSQL (single node), MySQL (single node)

CA Systems (Single Node)

CA systems are NOT distributed. They are single-node databases. Example: a single PostgreSQL instance. It has consistency and availability, but if that one server dies, the entire database is down. No partition tolerance because there are no partitions — only one node.

CP: Consistency + Partition Tolerance (MongoDB)

```
// —— MongoDB in CP mode ————————————————————————————
// You have 3 replica nodes: Primary, Secondary1, Secondary2
// Network partition: Primary isolated from both secondaries

// Write arrives at Primary:
// MongoDB cannot replicate to secondaries (network down)
// MongoDB REFUSES the write → loses Availability
// Why? To maintain Consistency — can't return "success"
// when it hasn't replicated

// Clients trying to write get: "Not Master" error
// System is unavailable until network heals
```

AP: Availability + Partition Tolerance (Cassandra)

```
// —— Cassandra in AP mode ————————————————————————————
// You have 3 nodes: A, B, C
// Network partition: A isolated from B and C

// Write "X" arrives at node A:
// A accepts write immediately → keeps Availability
// But B and C don't know about X yet

// Read from node B:
// B returns old value (hasn't seen X) → loses Consistency
// Eventually B learns about X (eventual consistency)

// System stays available, but data is temporarily inconsistent
```

Real-World Example: Bank Transfer

CP System (MongoDB)	AP System (Cassandra)
You transfer $100: debit A, credit B	You transfer $100: debit A, credit B
Network partition happens	Network partition happens
MongoDB cannot replicate to majority	Cassandra accepts both writes
MongoDB REFUSES transaction	Node A knows debit, Node B knows credit
User sees: "Transaction failed, try again"	Partition heals, conflict detected
Result: Annoying, but money is safe	Result: Inconsistent balances (DISASTER for banking)

Which to Choose?

CP (Consistency over Availability): Use when correctness is critical. Banking, payments, inventory. Better to be unavailable than wrong.

AP (Availability over Consistency): Use when uptime is critical and eventual consistency is acceptable. Social media likes, view counts, shopping cart (before checkout).

Most systems are NOT purely CP or AP — they offer tunable consistency levels.

Interview Answer: CAP Theorem

"CAP theorem states that in a distributed database, you can only guarantee two of three properties: Consistency, Availability, and Partition Tolerance. Partition tolerance is required for distributed systems, so the real choice is between CP and AP. CP systems like MongoDB prioritize consistency — they refuse requests during network partitions to avoid returning stale data. AP systems like Cassandra prioritize availability — they always respond but may return stale data. Choose CP for financial systems where correctness matters. Choose AP for social features where availability matters more than perfect consistency."

---

04 🔄 NORMALIZATION VS DENORMALIZATION

Data Duplication: When to Avoid It, When to Embrace It

🎯 The Core Trade-off

Normalization eliminates redundancy — each fact stored once. Denormalization duplicates data for faster reads. Normalization makes writes easy, reads slow (JOINs). Denormalization makes reads fast, writes complex.

Normalization (SQL)

Split data into multiple related tables. Link via foreign keys. Avoid duplication. Query with JOINs.

```
// ── Normalized Schema (3rd Normal Form) ──────────────────────

// Table: users
// | id | email        | name  |
// |----|---------------|-------|
// | 1  | alice@ex.com  | Alice |

// Table: posts
// | id | user_id | title        | content |
// |----|---------|---------------|---------|
// | 1  | 1       | "First Post"  | "..."   |
// | 2  | 1       | "Second Post" | "..."   |

// Table: comments
// | id | post_id | user_id | text     |
// |----|---------|---------|----------|
// | 1  | 1       | 1       | "Great!" |

// ☑ Pros:
// - No duplication: user name stored once
// - Update once: change Alice's name → updates everywhere
// - Data integrity: foreign keys enforce relationships

// ✕ Cons:
// - Must JOIN 3 tables to show post with author name and comments
// - Slow: every read requires complex JOINs

// Query to get post with author and comments:
SELECT posts.title, users.name, comments.text
FROM posts
JOIN users ON posts.user_id = users.id
JOIN comments ON comments.post_id = posts.id
WHERE posts.id = 1;

// 3 tables accessed, indexes traversed, rows joined
```

Denormalization (NoSQL)

Embed related data in one document. Duplicate data intentionally. Read with one query, no JOINs.

```
// ── Denormalized Schema (MongoDB) ────────────────────────────

// Collection: posts
{
  _id: ObjectId("abc123"),
  title: "First Post",
  content: "...",
  author: {              // ← Embedded user data
```

```
    id: 1,
    name: "Alice",
    email: "alice@ex.com"
  },
  comments: [             // ← Embedded comments
    {
      user: { id: 1, name: "Alice" },
      text: "Great!",
      created: "2025-01-01"
    }
  ]
}

// ☑ Pros:
// - One query: db.posts.findOne({ _id: ... })
// - Fast: no JOINs, no index traversal across tables
// - Scales: each document independent, easy to shard

// ✕ Cons:
// - Duplication: Alice's name copied into every post, every comment
// - Update everywhere: if Alice changes name, must update ALL posts
// - Risk of inconsistency: if update fails halfway, some docs stale
```

The Normalization Levels (SQL)

Normal Forms Explained

1st Normal Form (1NF)

Atomic values: no arrays or nested objects in columns. Each cell has one value.

2nd Normal Form (2NF)

1NF + no partial dependencies. Every non-key column depends on the WHOLE primary key, not part of it.

3rd Normal Form (3NF)

2NF + no transitive dependencies. Non-key columns depend only on the primary key, not on other non-key columns.

In practice: aim for 3NF in SQL, then selectively denormalize hot paths.

When to Normalize (SQL)

- Frequent updates: user profile data that changes often
- Data integrity: financial records where duplication risks inconsistency
- Complex queries: reporting, aggregations, GROUP BY analytics
- Write-heavy workloads: more writes than reads

When to Denormalize (NoSQL or SQL)

- Read-heavy workloads: blog posts read 1000x more than written
- Predictable access patterns: always need author name with post
- Performance critical: landing pages, dashboards — 10ms not 100ms
- Rare updates: product catalogs, reference data

Hybrid: Denormalize in SQL

You can denormalize in SQL too. Add redundant columns, use materialized views, cache computed values.

```
// ── Example: User post_count (denormalized) ──────────────────
CREATE TABLE users (
  id       SERIAL PRIMARY KEY,
  email    VARCHAR(255),
  name     VARCHAR(100),
  post_count INTEGER DEFAULT 0  -- ← Denormalized!
);

// Update count when post created:
BEGIN;
  INSERT INTO posts (user_id, title) VALUES (1, "New Post");
  UPDATE users SET post_count = post_count + 1 WHERE id = 1;
COMMIT;

// ☑ Pros: SELECT name, post_count FROM users → no JOIN needed
// ✕ Cons: must update two tables, risk of inconsistency

// Alternative: Use a materialized view (PostgreSQL):
CREATE MATERIALIZED VIEW user_post_counts AS
SELECT users.id, users.name, COUNT(posts.id) AS post_count
FROM users LEFT JOIN posts ON users.id = posts.user_id
GROUP BY users.id;

// Refresh periodically: REFRESH MATERIALIZED VIEW user_post_counts;
```

Interview Answer: Normalization vs Denormalization

"Normalization eliminates data duplication by splitting into related tables. This maintains consistency — updating once updates everywhere — but requires JOINs which are slow. Denormalization duplicates data for fast reads — everything in one document, no JOINs. The trade-off: normalized data is consistent but slow to read. Denormalized data is fast to read but risks inconsistency on updates. I normalize by default in SQL for data integrity, then selectively denormalize hot paths. In NoSQL like MongoDB, I embed frequently-accessed related data."

---

06 ⚡ QUERY OPTIMIZATION

Making Queries Fast: EXPLAIN, N+1, JOINs

🎯 The N+1 Query Problem

The #1 performance killer in backend applications. You fetch N items, then loop and query for each one. Result: 1 + N queries instead of 1 query.

```
// ── N+1 Problem (BAD) ──────────────────────────────────────
// Get all users:
const users = await User.findAll();  // Query 1

// For each user, get their posts:
for (const user of users) {
  user.posts = await Post.findAll({ where: { userId: user.id } });
  // Query 2, 3, 4, 5... 101 if 100 users!
}

// 100 users = 101 database queries
// 1000 users = 1001 queries → app grinds to a halt

// ── Solution: JOIN or eager loading ────────────────────────
// SQL: Use JOIN
SELECT users.*, posts.*
FROM users
LEFT JOIN posts ON users.id = posts.user_id;
// One query returns everything!
```

```
// ORM: Use eager loading (include/populate)
const users = await User.findAll({ include: Post });
// Sequelize generates JOIN automatically
// One query, all data
```

SELECT Only What You Need

```
// ❌ BAD: Fetch all columns
SELECT * FROM users;
// Returns: id, email, name, password_hash, bio, avatar_url, ...
// Wastes bandwidth, memory, network

// ☑ GOOD: Fetch only needed columns
SELECT id, name FROM users;
// Returns only what you actually use

// ORM example:
await User.findAll({ attributes: ['id', 'name'] });
```

Pagination: Always LIMIT

```
// ❌ NEVER do this:
SELECT * FROM posts ORDER BY created_at DESC;
// Returns ALL posts → 1 million rows → crashes app

// ☑ Always paginate:
SELECT * FROM posts
ORDER BY created_at DESC
LIMIT 20 OFFSET 0;  -- Page 1

LIMIT 20 OFFSET 20;  -- Page 2

// Better: Use cursor-based pagination for large datasets
SELECT * FROM posts
WHERE id < 12345  -- Last ID from previous page
ORDER BY id DESC
LIMIT 20;
```

Interview Answer: Query Optimization

*"Query optimization starts with avoiding N+1 queries — use JOINs or eager loading instead of loops. I always SELECT only needed columns, never SELECT *. For large result sets, I paginate with LIMIT and OFFSET, or cursor-based for better performance. I use EXPLAIN ANALYZE to check if queries use indexes. Slow queries are almost always missing indexes*

> *or doing table scans. I monitor slow query logs in production and add indexes to frequently queried columns."*

---

07 🔐 TRANSACTIONS

Ensuring Data Consistency Under Concurrent Load

🎯 What Are Transactions?

A transaction is a group of SQL statements that execute as a single unit. Either ALL succeed (commit), or ALL fail (rollback). This ensures data consistency.

```
// ── Transaction Example: Transfer Money ──────────────────────
BEGIN TRANSACTION;

  UPDATE accounts SET balance = balance - 100 WHERE user_id = 1;
  UPDATE accounts SET balance = balance + 100 WHERE user_id = 2;

COMMIT;  -- Success: both updates permanent

// If second UPDATE fails:
ROLLBACK;  -- Both updates undone
```

Using Transactions in Code

```
// Node.js with Sequelize:
const t = await sequelize.transaction();

try {
  await Account.update(
    { balance: sequelize.literal('balance - 100') },
    { where: { userId: 1 }, transaction: t }
  );

  await Account.update(
    { balance: sequelize.literal('balance + 100') },
    { where: { userId: 2 }, transaction: t }
  );

  await t.commit();  // Both succeed
```

```
} catch (error) {
  await t.rollback();  // One failed, undo both
  throw error;
}
```

When to Use Transactions

- Multi-step operations: order + payment + inventory update
- Financial operations: any money movement
- Critical data updates: user registration (create user + profile + settings)
- Preventing race conditions: checking stock before decrementing

Interview Answer: Transactions

"Transactions group multiple database operations into an atomic unit. They guarantee ACID properties — all operations succeed together or all roll back. I use transactions for financial operations like transfers, multi-step workflows like order creation, and anywhere data consistency is critical. In Node.js with Sequelize, I wrap operations in a transaction and handle errors with try-catch-rollback. The database isolation level determines how concurrent transactions interact."

---

08 📞 CONNECTION POOLING

Why Opening Connections Per Request Kills Your Database

🎯 The Problem

Opening a database connection is SLOW (50-100ms). If you open a new connection for every HTTP request, your app can only handle 10-20 requests per second. Connection pooling solves this.

```
// ❌ BAD: New connection per request
app.get('/users', async (req, res) => {
  const db = await connectToDatabase();  // 100ms!
  const users = await db.query('SELECT * FROM users');
  await db.close();
  res.json(users);
```

```
});

// Every request waits 100ms just to connect
// 10 concurrent requests = database overwhelmed

// ☑ GOOD: Connection pool
const pool = new Pool({ max: 20 });  // Reuse 20 connections

app.get('/users', async (req, res) => {
  const client = await pool.connect();  // <1ms (reused)
  const users = await client.query('SELECT * FROM users');
  client.release();  // Return to pool
  res.json(users);
});
```

How Connection Pooling Works

```
// Pool maintains N open connections:

Pool: [conn1, conn2, conn3, ..., conn20]
     idle   idle   busy      idle

// Request arrives:
// 1. Pool gives you an idle connection (instant)
// 2. You use it
// 3. You release it back to pool
// 4. Next request reuses same connection

// If all 20 busy: new requests wait in queue
```

Pool Configuration

```
const pool = new Pool({
  max: 20,            // Max connections
  min: 5,             // Always keep 5 open
  idleTimeoutMillis: 30000,  // Close idle after 30s
  connectionTimeoutMillis: 2000,  // Wait max 2s for connection
});
```

Interview Answer: Connection Pooling

"Connection pooling reuses database connections instead of opening new ones per request. Opening a connection takes 50-100ms, but reusing from a pool takes <1ms. The pool maintains a set of open connections — when a request needs one, it borrows from the pool and returns it when done. This prevents overwhelming the database with connection

> *overhead and dramatically improves throughput. I configure pool size based on concurrent load — typically 10-50 connections for most applications."*

---

09 🔄 DATABASE MIGRATIONS

Evolving Your Schema Safely in Production

🎯 What Are Migrations?

Migrations are versioned scripts that modify your database schema. They track changes over time and let you evolve the schema without manual SQL commands in production.

```
// ── Migration File: 001_create_users.js ──────────────────────
module.exports = {
  up: async (queryInterface, Sequelize) => {
    await queryInterface.createTable('users', {
      id: { type: Sequelize.INTEGER, primaryKey: true, autoIncrement: true },
      email: { type: Sequelize.STRING, unique: true, allowNull: false },
      name: { type: Sequelize.STRING },
      created_at: { type: Sequelize.DATE, defaultValue: Sequelize.NOW },
    });
  },

  down: async (queryInterface) => {
    await queryInterface.dropTable('users');
  }
};

// Run: npx sequelize-cli db:migrate
// Rollback: npx sequelize-cli db:migrate:undo
```

Adding a Column (Zero-Downtime)

```
// ── Migration: Add column with default ───────────────────────
module.exports = {
  up: async (queryInterface, Sequelize) => {
    // Step 1: Add column with NULL allowed
    await queryInterface.addColumn('users', 'phone', {
      type: Sequelize.STRING,
      allowNull: true,  // Important for existing rows!
```

```
    });

    // Step 2: Backfill existing rows (optional)
    await queryInterface.sequelize.query(
      'UPDATE users SET phone = '' WHERE phone IS NULL'
    );

    // Step 3: Make NOT NULL (separate migration, after deploy)
    // This prevents locking the table during backfill
  },

  down: async (queryInterface) => {
    await queryInterface.removeColumn('users', 'phone');
  }
};
```

Best Practices

- Never modify old migrations: create new ones
- Always test locally first: run migration on dev DB
- Idempotent migrations: safe to run multiple times
- Small incremental changes: easier to rollback
- Zero-downtime: add column as nullable, backfill, then make NOT NULL in separate migration

Interview Answer: Migrations

"Migrations are versioned schema change scripts that track database evolution. Instead of manually running ALTER TABLE in production, I write migration files with up and down methods. Migrations run in order, and each is recorded in a migrations table to prevent re-running. For zero-downtime changes, I add new columns as nullable first, deploy code that handles both old and new schema, backfill data, then make the column NOT NULL in a followup migration. This prevents locking tables during deploys."

Final Interview Cheatsheet

Here is your complete database knowledge summary for interviews:

Topic	Key Takeaway
SQL vs NoSQL	SQL for complex queries and ACID. NoSQL for flexible schema and horizontal scale.
ACID	Atomicity (all or nothing), Consistency (rules enforced), Isolation (no interference), Durability (survives crashes).
CAP Theorem	Pick 2 of 3: Consistency, Availability, Partition Tolerance. CP for banking, AP for social features.
Normalization	3NF eliminates duplication. Denormalize selectively for read performance.
Indexing	B-tree indexes turn O(n) scans into O(log n) lookups. Index WHERE clauses and foreign keys.
Query Optimization	Avoid N+1 queries. SELECT only needed columns. Always paginate. Use EXPLAIN ANALYZE.
Transactions	Group operations atomically. Essential for financial ops and multi-step workflows.
Connection Pooling	Reuse connections (1ms) instead of opening new ones (100ms). Pool size = concurrent load.
Migrations	Version control for schema. Never modify old migrations. Zero-downtime = nullable first.

Master these fundamentals and you can design, optimize, and debug any database system.